

Homework 2 (100pt)

1. We know C is not type-safe and we have shown (see slides) how to manipulate the value of some variable without explicitly reference its name in the code. In fact, we can even manipulate the code to change the control flow without explicitly writing function calls or returns. Look at the following C code.

```
#include<stdio.h>
void fun2(int a) {
    printf("a=%d\n", a);
}
void fun1() {
    fun2(7083); /* <== Some code is needed here to replace
"fun2(7083)" */
}
int main() {
    fun1();
}
```

We see main() calls fun1(), which then calls fun2(7083). The result is "a=7083". Now your job is to change the code in fun1() such that it will run fun2() with argument 7083, but without actually writing "fun2(7083)" in the code or accepting any user input. Your code in fun1() is expected to be a set of pointer operations without any function call. (50pts)

Write and test your code. Submit your code, screen-shot and explain the results.

Hints: (1) You need to remember how cdecl works; (2) objdump helps a lot; (3) compile your code using gcc options -O0 and -m32. -O0 means no optimization, so we have a simple stack layout. -m32 means forcing 32-bit code that by default uses the cdecl calling convention. (4) If your machine is 64-bit, you can try to install 32-bit gcc support using command "apt-get install gcc-multilib" in Ubuntu.

2. In class, we gave a very basic macro design to implement stack guard with canary. In that design, we haven't talk about how to define __SECRET_VAR in the macros. A simple way to define __SECRET_VAR is some marco like

```
#define __SECRET_VAR 11223344
```

But the issue is that it is a constant so an attacker can know. A better strategy is to make it a variable and assign a random number to it at run time.

You job is to write the entire set of macros in a head file “mycanary.h” so a user’s program can include the header file and use all macros for stack protection. You have already been given the following user’s code, which uses four macros:

DEF_CANARY(), PLACE_CANARY(), SAFE_RETURN(), and INIT_CANARY().

Please complete all four macros in the header file “mycanary.h” so __SECRET_VAR can be assigned randomly at run time. Submit the content of your “mycanary.h”.

Screen-shot the result to show your macros can successfully detect buffer overflow in the following code. (50pts)

```
#include<stdio.h>
#include<string.h>
#include"mycanary.h"

DEF_CANARY(); /* define canary as global*/

void vul_fun(char *str) {
    PLACE_CANARY(); /* place a canary */
    char buf[16];
    strcpy(buf, str);
    printf("buf=%s\n", buf);
    SAFE_RETURN(); /* check canary and return */
}

int main(){
    INIT_CANARY(); /* randomize the value of canary */
    vul_fun("123456789012345678901234567890");
}
```

Hints: You can define a global variable in DEF_CANARY, then assign a random value (using random functions in C) to __SECRET_VAR in INIT_CANARY. PLACE_CANARY() and SAFE_RETURN() can be found in slides.